

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт

Про виконання лабораторної роботи №6
«Використання лямбда-виразів»

Виконала:

Студентка групи ФЕП-11

Матла Діана Ярославівна

Перевірив:

Асистент Левуш Павло Назарович

Львів 2026

Мета: Метою лабораторної роботи є вивчення синтаксису та можливостей використання лямбда-виразів у мові програмування C++. Дослідити способи створення анонімних функцій, особливості захоплення змінних із зовнішньої області видимості, а також застосування лямбда-виразів для реалізації математичних обчислень.

Хід роботи

Файли та готовий код було відправлено на github за посиланням:

<https://github.com/Diana-Matla/OOP/tree/main/lab6>

У ході виконання лабораторної роботи було розглянуто та реалізовано використання лямбда-виразів для розв'язання прикладних математичних задач.

Реалізовано обчислення коренів квадратного рівняння за допомогою лямбда-виразів для знаходження дискримінанта та коренів рівняння.

Розроблено програмну реалізацію чисельного методу бісекції (ділення відрізка навпіл), у якій математична функція задана у вигляді лямбда-виразу.

Реалізовано метод Ньютона для знаходження кореня нелінійного рівняння, де функція та її похідна (обчислена чисельно) також задані через лямбда-вирази із використанням захоплення змінних.

Досліджено особливості захоплення змінних за значенням та за посиланням, а також вплив цього механізму на поведінку алгоритмів.

```
C Dyhotomia_class.h > ...
1  #ifndef DYHOTOMIA_CLASS_H
2  #define DYHOTOMIA_CLASS_H
3
4  class Dyhotomia_class {
5  private:
6      double a, b, eps;
7
8  public:
9      Dyhotomia_class();
10     ~Dyhotomia_class();
11
12     void setVolumes(double vol_a, double vol_b);
13     void setTolerance(double vol_eps);
14
15     double Dyhotomia();
16     double Newton();
17 };
18
19 void quadratic(double a, double b, double c);
20
21 #endif
```

Dyhotomia.h

```
main.cpp > main()
1  #include "Dyhotomia_Class.h"
2  #include <iostream>
3
4  using namespace std;
5
6  int main() {
7
8      cout << "QUADRATIC EQUATION \n";
9      quadratic(1, -5, 6);
10
11      Dyhotomia_class solver;
12
13      solver.setVolumes(1, 2);
14      solver.setTolerance(0.0001);
15
16      cout << "\nBISECTION METHOD\n";
17      cout << "Root: " << solver.Dyhotomia() << endl;
18
19      cout << "\nNEWTON METHOD\n";
20      cout << "Root: " << solver.Newton() << endl;
21
22      return 0;
23 }
```

```
Dyhotomia_class.cpp > Newton()
1  #include "Dyhotomia_Class.h"
2  #include <iostream>
3  #include <cmath>
4
5  using namespace std;
6
7  Dyhotomia_class::Dyhotomia_class() {
8      a = 0;
9      b = 0;
10     eps = 0;
11 }
12
13 Dyhotomia_class::~Dyhotomia_class() {}
14
15 void Dyhotomia_class::setVolumes(double vol_a, double vol_b) {
16     a = vol_a;
17     b = vol_b;
18 }
19
20 void Dyhotomia_class::setTolerance(double vol_eps) {
21     eps = vol_eps;
22 }
23
```

Main.cpp

```
double Dyhotomia_class::Dyhotomia() {

    auto f = [](double x) -> double {
        if (fabs(x) < 1e-12)
            return NAN;
        return cos(2 / x) - 2 * sin(1 / x) + 1 / x;
    };

    double left = a;
    double right = b;

    double f_left = f(left);
    double f_right = f(right);

    if (isnan(f_left) || isnan(f_right)) {
        cout << "Function undefined in interval" << endl;
        return NAN;
    }

    if (f_left * f_right > 0) {
        cout << "No root in interval" << endl;
        return NAN;
    }

    if (fabs(f_left) < eps) return left;
    if (fabs(f_right) < eps) return right;

    double mid, f_mid;
```

```
    while (fabs(right - left) >= eps) {

        mid = (left + right) / 2.0;
        f_mid = f(mid);

        if (fabs(f_mid) < eps)
            return mid;

        if (f_left * f_mid < 0) {
            right = mid;
            f_right = f_mid;
        } else {
            left = mid;
            f_left = f_mid;
        }
    }

    return (left + right) / 2.0;
}
```

```
double Dyhotomia_class::Newton() {

    auto f = [](double x) -> double {
        if (fabs(x) < 1e-12)
            return NAN;
        return cos(2 / x) - 2 * sin(1 / x) + 1 / x;
    };

    auto df = [&](double x) -> double {
        double h = 1e-6;
        return (f(x + h) - f(x - h)) / (2 * h);
    };

    double x = (a + b) / 2.0;

    if (fabs(f(x)) < eps)
        return x;

    for (int i = 0; i < 100; i++) {

        double dfx = df(x);

        if (fabs(dfx) < 1e-12) {
            cout << "Derivative too small" << endl;
            return NAN;
        }
    }
```

```

        double x_new = x - f(x) / dfx;

        if (fabs(f(x_new)) < eps || fabs(x_new - x) < eps)
            return x_new;

        x = x_new;
    }

    cout << "Newton did not converge" << endl;
    return x;
}

void quadratic(double a, double b, double c) {

    auto D = [](double a, double b, double c) {
        return b * b - 4 * a * c;
    };

    double d = D(a, b, c);

    if (a == 0) {
        cout << "Not quadratic equation\n";
        return;
    }

    if (d < 0) {
        cout << "No real roots\n";
        return;
    }

    auto x1 = [](double a, double b, double d) {
        return (-b + sqrt(d)) / (2 * a);
    };

    auto x2 = [](double a, double b, double d) {
        return (-b - sqrt(d)) / (2 * a);
    };

    if (d == 0) {
        cout << "x = " << x1(a, b, d) << endl;
    } else {
        cout << "x1 = " << x1(a, b, d) << endl;
        cout << "x2 = " << x2(a, b, d) << endl;
    }
}

```

Вивід з терміналу

```

PS D:\labs\OOP\lab 6> cd "d:\lab
QUADRATIC EQUATION
x1 = 3
x2 = 2

BISECTION METHOD
Root: 1.87549

NEWTON METHOD
Root: 1.87561
PS D:\labs\OOP\lab 6>

```

Висновок: У ході лабораторної роботи було досліджено використання лямбда-виразів у C++ як інструменту для створення анонімних функцій. Було

реалізовано їх застосування в задачах чисельного аналізу, зокрема для розв'язання квадратного рівняння, а також знаходження коренів нелінійних рівнянь методами бісекції та Ньютона.

Отримані результати показали, що використання лямбда-виразів дозволяє зробити код більш компактним, зрозумілим та локалізованим. Також було підтверджено ефективність використання захоплення змінних для реалізації гнучких алгоритмів.